BABU BANARASI DAS UNIVERSITY

2026-2027



SCHOOL OF COMPUTER APPLICATION

No SQL project on

Student Database Using MongoDB

Submitted to:
Mr. Harendra Singh
Department: BCA DS & Al.
Date: 6 / 10 / 2026.

Submitted by: Suhani
Class: BCA DS & AI
Section: 26
Roll no: 1250258460
Sub: NoSQL Project

ACKNOWLEDGEMENT

I am thankful to my teacher for giving me this project on NoSQL and MongoDB. While making this project, I learned how student data can be stored and managed in a NoSQL database. I also learned different MongoDB operations such as inserting, searching, updating, deleting, sorting and counting data. I am also thankful to my teacher and everyone who helped me in completing this project.

INTRODUCTION

NoSQL means Not Only SQL. It is used for storing and managing data in a flexible way. MongoDB is a popular NoSQL database. It stores data in documents instead of traditional rows and columns.

In this project, a student database is made using MongoDB. The database contains 50 student records. Each student record has details like ID, name, age, course, marks and city. Different MongoDB operations are used on this data, such as create, insert, find, greater than, less than, equal to, update, delete, sort and count.

The main aim of this project is to understand these operations with simple practical examples.

50 STUDENT DATA USED IN THE PROJECT

ID	Name	Age	Course	Marks	City
1	Aarav	19	BCA	57	Kannauj
2	Aditi	20	BSc	64	Kanpur
3	Akash	21	BBA	71	Delhi
4	Ananya	22	BCA	78	Agra
5	Arjun	18	BCA	85	Lucknow
6	Bhavna	19	BCA	92	Kannauj
7	Chetan	20	BSc	99	Kanpur
8	Deepak	21	BBA	55	Delhi
9	Diya	22	BCA	62	Agra
10	Esha	18	BCA	69	Lucknow
11	Farhan	19	BCA	76	Kannauj
12	Gauri	20	BSc	83	Kanpur
13	Harsh	21	BBA	90	Delhi
14	Isha	22	BCA	97	Agra
15	Jatin	18	BCA	53	Lucknow
16	Kajal	19	BCA	60	Kannauj
17	Karan	20	BSc	67	Kanpur
18	Kavya	21	BBA	74	Delhi
19	Laksh	22	BCA	81	Agra
20	Meera	18	BCA	88	Lucknow
21	Naman	19	BCA	95	Kannauj
22	Neha	20	BSc	51	Kanpur
23	Om	21	BBA	58	Delhi
24	Pooja	22	BCA	65	Agra
25	Pranav	18	BCA	72	Lucknow
26	Priya	19	BCA	79	Kannauj
27	Rahul	20	BSc	86	Kanpur
28	Riya	21	BBA	93	Delhi
29	Rohan	22	BCA	100	Agra
30	Sakshi	18	BCA	56	Lucknow
31	Sameer	19	BCA	63	Kannauj
32	Sanya	20	BSc	70	Kanpur
33	Shivam	21	BBA	77	Delhi
34	Shruti	22	BCA	84	Agra
35	Siddharth	18	BCA	91	Lucknow
36	Simran	19	BCA	98	Kannauj
37	Tanmay	20	BSc	54	Kanpur
38	Tanya	21	BBA	61	Delhi
39	Uday	22	BCA	68	Agra
40	Vansh	18	BCA	75	Lucknow
41	Varun	19	BCA	82	Kannauj
42	Ved	20	BSc	89	Kanpur
43	Vikas	21	BBA	96	Delhi
44	Yash	22	BCA	52	Agra
45	Yashi	18	BCA	59	Lucknow
46	Zoya	19	BCA	66	Kannauj
47	Abhay	20	BSc	73	Kanpur

48	Anjali	21	BBA	80	Delhi
49	Dev	22	BCA	87	Agra
50	Muskan	18	BCA	94	Lucknow

1. Create Database and Collection

Operation / Formula

```
use StudentDB

db.createCollection("students")
```

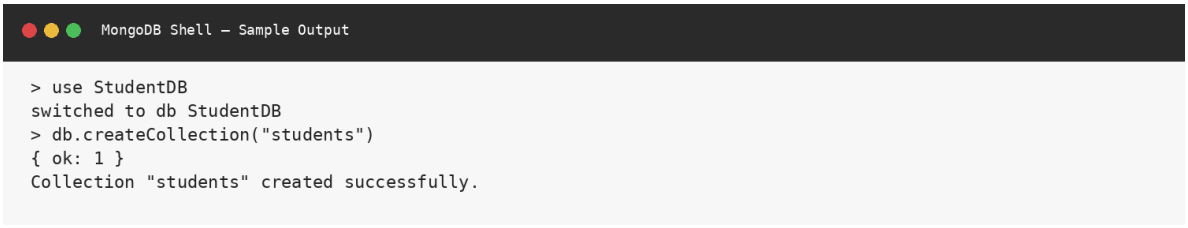
Explanation

The use command selects the database and createCollection() creates the students collection.

How it works

First StudentDB is selected. Then a collection named students is created.

Output



2. Insert 50 Students Data

Operation / Formula

```
db.students.insertMany([

  { _id:1, name:"Aarav", age:19, course:"BCA", marks:57, city:"Lucknow" },

  { _id:2, name:"Aditi", age:20, course:"BCA", marks:64, city:"Kannauj" },

  // ... 48 more student documents ...

])
```

Explanation

insertMany() is used to insert many student documents at the same time. Here, 50 student records are inserted.

How it works

If the command runs successfully, MongoDB returns acknowledged: true and insertedCount: 50.

Output

MongoDB Shell — Sample Output

```
> db.students.insertMany([ ... 50 student documents ... ])
{ acknowledged: true, insertedCount: 50 }
> db.students.countDocuments()
50
```

3. Display / Read Student Data

Operation / Formula

```
db.students.find(

  {},

  { _id:1, name:1, marks:1 }

).limit(5)
```

Explanation

find() is used to display documents from the collection. Projection is used to show only selected fields.

How it works

The query displays the selected information of the first five students.

Output

MongoDB Shell – Sample Output

```
> db.students.find({}, { _id:1, name:1, marks:1 }).limit(5)
{ _id: 1, name: "Aarav", marks: 57 }
{ _id: 2, name: "Aditi", marks: 64 }
{ _id: 3, name: "Akash", marks: 71 }
{ _id: 4, name: "Ananya", marks: 78 }
{ _id: 5, name: "Arjun", marks: 85 }
```

4. Greater Than ($gt)

Operation / Formula

```
db.students.find(

  { marks: { $gt: 80 } },

  { name:1, marks:1, _id:0 }

)
```

Explanation

$gt means greater than. It shows students whose marks are more than 80.

How it works

Only records having marks greater than 80 are displayed.

Output

MongoDB Shell — Sample Output

```
> db.students.find({ marks: { $gt: 80 } }, { name:1, marks:1, _id:0 })
{ name: "Arjun", marks: 85 }
{ name: "Bhavna", marks: 92 }
{ name: "Chetan", marks: 99 }
{ name: "Gauri", marks: 83 }
{ name: "Harsh", marks: 90 }
{ name: "Isha", marks: 97 }
... only records with marks > 80 are returned ...
```

5. Less Than ($lt)

Operation / Formula

```
db.students.find(

  { marks: { $lt: 60 } },

  { name:1, marks:1, _id:0 }

)
```

Explanation

$lt means less than. It shows students whose marks are below 60.

How it works

Only records having marks less than 60 are displayed.

Output

MongoDB Shell – Sample Output

```
> db.students.find({ marks: { $lt: 60 } }, { name:1, marks:1, _id:0 })
{ name: "Aarav", marks: 57 }
{ name: "Deepak", marks: 55 }
{ name: "Jatin", marks: 53 }
{ name: "Neha", marks: 51 }
{ name: "Om", marks: 58 }
{ name: "Sakshi", marks: 56 }
... only records with marks < 60 are returned ...
```

6. Equal To ($eq)

Operation / Formula

```
db.students.find(

  { marks: { $eq: 71 } },

  { name:1, marks:1, _id:0 }

)
```

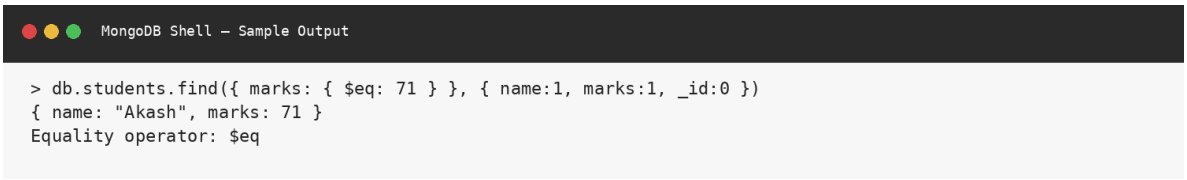
Explanation

$eq means equal to. It finds students whose marks are exactly 71.

How it works

Only records having marks equal to 71 are displayed.

Output



7. Update Student Data

Operation / Formula

```
db.students.updateOne(
  { _id: 1 },
  { $set: { marks: 95 } }
)
```

Explanation

updateOne() changes the value of a field in one matching document. $set is used to give the new value.

How it works

The student with ID 1 is found and the marks field is changed to 95.

Output

MongoDB Shell – Sample Output

```
> db.students.updateOne({ _id: 1 }, { $set: { marks: 95 } })
{ acknowledged: true, matchedCount: 1, modifiedCount: 1 }
> db.students.findOne({ _id: 1 })
{ _id: 1, name: "Aarav", marks: 95, ... }
```

8. Delete Student Data

Operation / Formula

```
db.students.deleteOne(
  { _id: 50 }
)
```

Explanation

deleteOne() removes one document that matches the given condition.

How it works

The document with ID 50 is deleted. deletedCount: 1 confirms the deletion.

Output

MongoDB Shell – Sample Output

```
> db.students.deleteOne({ _id: 50 })
{ acknowledged: true, deletedCount: 1 }
> db.students.countDocuments()
49
```

9. Sort Student Data

Operation / Formula

```
db.students.find(

  {},

  { name:1, marks:1, _id:0 }

).sort({ marks: -1 }).limit(5)
```

Explanation

sort() arranges the records. 1 is ascending order and -1 is descending order.

How it works

The students are arranged from highest marks to lowest marks and the first five are shown.

Output

MongoDB Shell – Sample Output

```
> db.students.find({}, { name:1, marks:1, _id:0 }).sort({ marks: -1 }).limit(5)
{ name: "Rohan", marks: 100 }
{ name: "Chetan", marks: 99 }
{ name: "Simran", marks: 98 }
{ name: "Isha", marks: 97 }
{ name: "Vikas", marks: 96 }
Sorted in descending order of marks.
```

10. Count Documents

Operation / Formula

```
db.students.countDocuments(
  { marks: { $gte: 60 } }
)
```

```
db.students.countDocuments(
  { course: "BCA" }
)
```

Explanation

countDocuments() counts the number of documents that match a condition.

How it works

The command gives the number of students satisfying the given condition.

Output

MongoDB Shell – Sample Output

```
> db.students.countDocuments({ marks: { $gte: 60 } })
31
> db.students.countDocuments({ course: "BCA" })
30
CountDocuments() returns the number of documents matching a condition.
```

CONCLUSION

In this project, I used MongoDB to store and manage 50 student records. I performed different operations such as insert, find, comparison, update, delete, sort and count. These operations helped me understand how data is handled in a NoSQL database. MongoDB is useful because its document format is flexible and easy to work with.